

JOBSHEET V
BRUTE FORCE DAN DIVIDE CONQUER



Disusun oleh:

Nama : Galih Candra Kirana
No. Absen : 10
Kelas : TI 1G
NIM : 254107020080

POLITEKNIK NEGERI MALANG
Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141
TAHUN 2025-2026

5.1 Tujuan Praktikum

Setelah melakukan materi praktikum ini, mahasiswa mampu:

1. Mahasiswa mampu membuat algoritma bruteforce dan divide-conquer
2. Mahasiswa mampu menerapkan penggunaan algoritma bruteforce dan divide-conquer

5.2 Menghitung Nilai Faktorial dengan Algoritma Brute Force dan Divide and Conquer

5.2.1 Langkah-langkah Percobaan

Class Faktorial:

```
package Jobsheet5.BruteForceDivideConquer;
```

```
public class Faktorial {
```

```
    int faktorialBF(int n) {  
        int fakto = 1;  
        for (int i = 1; i <= n; i++) {  
            fakto = fakto * i;  
        }  
        return fakto;  
    }  
}
```

```
    int faktorialDC(int n) {  
        if (n == 1) {  
            return 1;  
        } else {  
            int fakto = n * faktorialDC(n - 1);  
            return fakto;  
        }  
    }  
}
```

Class MainFaktorial:

```
package Jobsheet5.BruteForceDivideConquer;
```

```

import java.util.Scanner;

public class MainFaktorial {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Masukkan nilai: ");

        int nilai = input.nextInt();

        Faktorial fk = new Faktorial();

        System.out.println("Nilai faktorial " + nilai + " menggunakan BF: " +
            fk.faktorialBF(nilai));

        System.out.println("Nilai faktorial " + nilai + " menggunakan DC: " +
            fk.faktorialDC(nilai));

        input.close();
    }
}

```

5.2.2 Verifikasi Hasil Percobaan

```

Masukkan nilai: 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120

```

5.2.3 Pertanyaan:

- 1 Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!
Jawaban:
Pada blok kode bagian if adalah base line yang digunakan untuk menghentikan proses rekursi ketika $n = 1$, yang akan mereturn nilai 1, Sedangkan blok kode else adalah proses rekursi yang memanggil dirinya dengan nilai lebih kecil ($n - 1$).

- 2 Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!

Jawaban:

Sangat memungkinkan, bukti:

```

public int faktorialBF(int n) {
    int fakto = 1;
    int i = 1;
    while (i <= n) {
        fakto = fakto * i;
        i++;
    }
    return fakto;
}

```

- 3 Jelaskan perbedaan antara fakto *= i; dan int fakto = n * faktorialDC(n-1); !

Jawaban:

Perbedaanya, fakto *= i digunakan pada metode iteratif dengan perulangan, sedangkan int fakto = n * faktorialDC(n-1) digunakan pada metode rekursif dengan memanggil fungsi dirinya sendiri hingga mencapai base case.

- 4 Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()

Jawban:

Kesimpulannya, Method faktorialBF() menghitung faktorial menggunakan perulangan (iteratif), sedangkan faktorialDC() menghitung faktorial menggunakan rekursif yang memanggil fungsi faktorial dengan nilai yang lebih kecil hingga mencapai base case.

5.2 Menghitung Hasil Pangkat dengan Algoritma Brute Force dan Divide and Conquer

5.2.1 Langkah-langkah Percobaan

Class Pangkat:

```
package Jobsheet5.BruteForceDivideConquer;
```

```
public class Pangkat {
```

```
    int nilai;
```

```
    int pangkat;
```

```
    Pangkat(int n, int p) {
```

```
        nilai = n;
```

```
        pangkat = p;
```

```
    }
```

```
    int PangkatBF(int a, int n) {
```

```
        int hasil = 1;
```

```
        for (int i = 0; i < n; i++) {
```

```
            hasil = hasil * a;
```

```
        }
```

```
        return hasil;
```

```
    }
```

```
    int pangkatDC(int a, int n) {
```

```

    if (n == 1) {
        return a;
    } else {
        if (n % 2 == 1) {
            return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2) * a);
        } else {
            return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2));
        }
    }
}
}
}
}

```

Class MainPangkat:

```

package Jobsheet5.BruteForceDivideConquer;
import java.util.Scanner;

public class MainPangkat {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Masukkan jumlah elemen: ");
        int elemen = input.nextInt();

        Pangkat[] png = new Pangkat[elemen];
        for (int i = 0; i < png.length; i++) {
            System.out.print("Masukkan nilai basis elemen ke-" + (i + 1) + " : ");
            int basis = input.nextInt();
            System.out.print("Masukkan nilai pangkat elemen ke-" + (i + 1) + " : ");
            int pangkat = input.nextInt();
            png[i] = new Pangkat(basis, pangkat);
        }
    }
}

```

```

        System.out.println("HASIL PANGKAT BRUTEFORCE");
        for (Pangkat p : png) {
            System.out.println(p.nilai + "^" + p.pangkat + ": " +
                p.PangkatBF(p.nilai, p.pangkat));
        }

        System.out.println("HASIL PANGKAT DIVIDE AND CONQUER");
        for (Pangkat p : png) {
            System.out.println(p.nilai + "^" + p.pangkat + ": " +
                p.pangkatDC(p.nilai, p.pangkat));
        }
        input.close();
    }
}

```

5.2.2 Verifikasi hasil percobaan

```

Masukkan jumlah elemen: 3
Masukkan nilai basis elemen ke-1 : 2
Masukkan nilai pangkat elemen ke-1 : 3
Masukkan nilai basis elemen ke-2 : 4
Masukkan nilai pangkat elemen ke-2 : 5
Masukkan nilai basis elemen ke-3 : 6
Masukkan nilai pangkat elemen ke-3 : 7
HASIL PANGKAT BRUTEFORCE
2^3: 8
4^5: 1024
6^7: 279936
HASIL PANGKAT DIVIDE AND CONQUER
2^3: 8
4^5: 1024
6^7: 279936

```

5.2.3 Pertanyaan

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!

Jawaban:

Perbedaannya, pangkatBF() menghitung pangkat dengan perulangan (iteratif), sedangkan pangkatDC() melakukan perkalian dengan dirinya sendiri secara rekursif.

2. Apakah tahap *combine* sudah termasuk dalam kode tersebut? Tunjukkan!

Jawaban:

Sudah, yaitu pada bagian:

```
return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2) * a);
```

```
return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2));
```

3. Pada method `pangkatBF()` terdapat parameter untuk melewati nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class `Pangkat` telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method `pangkatBF()` yang tanpa parameter?

Jawaban:

Masih relevan karena nilai yang akan dipangkatkan (*a*) dan nilai pangkat (*n*) bisa langsung diberikan saat method dipanggil tanpa harus bergantung pada atribut di dalam class.

Method tersebut bisa dibuat tanpa parameter dengan cara mengambil atribut pangkat. Contohnya seperti ini:

```
int PangkatBF() {  
    int hasil = 1;  
    for (int i = 0; i < this.pangkat; i++) {  
        hasil = hasil * this.nilai;  
    }  
    return hasil;  
}
```

4. Tarik tentang cara kerja method `pangkatBF()` dan `pangkatDC()`!

Jawaban:

Method `pangkatBF()` menghitung pangkat dengan perulangan (iteratif), sedangkan `pangkatDC()` menghitung pangkat secara rekursif dengan memecah perhitungan menjadi bagian yang lebih kecil.

5.2 Menghitung Sum Array dengan Algoritma Brute Force dan Divide and Conquer

5.2.1 Langkah-langkah Percobaan

Class `Sum`:

```
package Jobsheet5.BruteForceDivideConquer;
```

```
public class Sum {  
    double keuntungan[];  
  
    Sum(int el) {  
        keuntungan = new double[el];  
    }  
  
    double totalBF() {
```

```

double total = 0;
for (int i = 0; i < keuntungan.length; i++) {
    total = total + keuntungan[i];
}
return total;
}

```

```

double totalDC(double arr[], int l, int r) {
    if (l == r) {
        return arr[l];
    }
}

```

```

int mid = (l + r) / 2;
double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid + 1, r);
return lsum + rsum;
}
}

```

Class MainSum:

```

package Jobsheet5.BruteForceDivideConquer;
import java.util.Scanner;

public class MainSum {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan jumlah elemen: ");
        int elemen = input.nextInt();

        Sum sm = new Sum(elemen);
    }
}

```

```

        for (int i = 0; i < elemen; i++) {

            System.out.print("Masukkan keuntungan ke-" + (i + 1) + ": ");

            sm.keuntungan[i] = input.nextDouble();

        }

        System.out.println("Total keuntungan menggunakan Bruteforce: " +
sm.totalBF());

        System.out.println("Total keuntungan menggunakan Divide and Conquer:
" + sm.totalDC(sm.keuntungan, 0, elemen - 1));

        input.close();

    }

}

```

5.2.2 Verifikasi Hasil Percobaan

```

Masukkan jumlah elemen: 5
Masukkan keuntungan ke-1: 10
Masukkan keuntungan ke-2: 20
Masukkan keuntungan ke-3: 30
Masukkan keuntungan ke-4: 40
Masukkan keuntungan ke-5: 50
Total keuntungan menggunakan Bruteforce: 150.0
Total keuntungan menggunakan Divide and Conquer: 150.0

```

5.2.3 Pertanyaan

1. Kenapa dibutuhkan variable mid pada method TotalDC()?

Jawaban:

Variabel mid digunakan untuk menampung nilai tengah dari data, yang kemudian batas pembagian antara bagian kiri dan kanan.

2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

```

double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid+1, r);

```

Jawaban:

Statement tersebut dilakukan untuk memanggil totalDC secara rekursif dengan membagi array menjadi 2 bagian, yaitu kiri dan kanan.

3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

Jawaban:

Karena untuk menggabungkan hasil dari perhitungan bagian kiri dan kanan menjadi total keseluruhan.

4. Apakah base case dari totalDC()?

Jawaban:

Base case dari totalDC adalah saat bagian kiri(l) sama dengan bagian kanan(r), yaitu hanya ada satu elemen array yang diproses.

5. Tarik Kesimpulan tentang cara kerja totalDC()

Jawaban:

Method tersebut bekerja dengan metode divide and conquer, yaitu membagi array menjadi 2 bagian (l & r) secara rekursif hingga mencapai base case ($l == r$). Setelah itu, hasil dari masing-masing bagian digabungkan untuk mendapatkan total seluruh elemen array.

5.2 Latihan Praktikum

1) Informasi Dosen.

a. Class DaftarNilai:

```
package Jobsheet5.BruteForceDivideConquer;
```

```
public class DaftarNilai {
```

```
    String nama;
```

```
    String nim;
```

```
    int tahunMasuk;
```

```
    float nilaiUTS;
```

```
    float nilaiUAS;
```

```
    DaftarNilai() {
```

```
    }
```

```
    DaftarNilai(String nama, String nim, int tahunMasuk, float nilaiUTS, float  
    nilaiUAS) {
```

```
        this.nama = nama;
```

```
        this.nim = nim;
```

```
        this.tahunMasuk = tahunMasuk;
```

```
        this.nilaiUTS = nilaiUTS;
```

```
        this.nilaiUAS = nilaiUAS;
```

```
    }
```

```
    float nilUTSTertinggi(DaftarNilai nilai[], int indexMin, int indexMax) {
```

```

    if (indexMin == indexMax) {
        return nilai[indexMin].nilaiUTS;
    }

    int mid = (indexMin + indexMax) / 2;
    float leftNil = nilUTSTertinggi(nilai, indexMin, mid);
    float rightNil = nilUTSTertinggi(nilai, mid + 1, indexMax);

    if (leftNil > rightNil) {
        return leftNil;
    } else {
        return rightNil;
    }
}

float nilUTSTerendah(DaftarNilai nilai[], int indexMin, int indexMax) {
    if (indexMin == indexMax) {
        return nilai[indexMin].nilaiUTS;
    }

    int mid = (indexMin + indexMax) / 2;
    float leftNil = nilUTSTerendah(nilai, indexMin, mid);
    float rightNil = nilUTSTerendah(nilai, mid + 1, indexMax);

    if (leftNil < rightNil) {
        return leftNil;
    } else {
        return rightNil;
    }
}

```

```

float rata2UAS(DaftarNilai[] nilai) {
    float total = 0;
    for (int i = 0; i < nilai.length; i++) {
        total += nilai[i].nilaiUAS;
    }
    return total / nilai.length;
}
}

```

b. Class MainDaftarNilai:

```

package Jobsheet5.BruteForceDivideConquer;

public class MainDaftarNilai {
    public static void main(String[] args) {
        DaftarNilai[] dataNilai = {
            new DaftarNilai("Ahmad", "220101001", 2022, 78, 82),
            new DaftarNilai("Budi", "220101002", 2022, 85, 88),
            new DaftarNilai("Cindy", "220101003", 2021, 90, 87),
            new DaftarNilai("Dian", "220101004", 2021, 76, 79),
            new DaftarNilai("Eko", "220101005", 2023, 92, 95),
            new DaftarNilai("Fajar", "220101006", 2020, 88, 85),
            new DaftarNilai("Gina", "220101007", 2023, 80, 83),
            new DaftarNilai("Hadi", "220101008", 2020, 82, 84)
        };

        System.out.println("=====
=====");

        System.out.printf("%-10s %-12s %-12s %-10s %-10s\n",
            "Nama", "NIM", "TahunMasuk", "UTS", "UAS");
        for (int i = 0; i < dataNilai.length; i++) {

```

```

        System.out.printf("%-10s %-12s %-12d %-10.2f %-10.2f\n",
            dataNilai[i].nama,
            dataNilai[i].nim,
            dataNilai[i].tahunMasuk,
            dataNilai[i].nilaiUTS,
            dataNilai[i].nilaiUAS);
    }

    System.out.println("=====
=====");

    DaftarNilai nilai = new DaftarNilai();

    System.out.println("Nilai UTS Tertinggi: " + nilai.nilUTSTertinggi(dataNilai,
0, dataNilai.length - 1));

    System.out.println("Nilai UTS Terendah: " +
nilai.nilUTSTerendah(dataNilai, 0, dataNilai.length - 1));

    System.out.println("Nilai rata-rata UAS: " + String.format("%.2f",
nilai.rata2UAS(dataNilai)));

    }

}

```

c. Output

```

=====
Nama      NIM      TahunMasuk  UTS      UAS
Ahmad     220101001  2022       78.00    82.00
Budi      220101002  2022       85.00    88.00
Cindy     220101003  2021       90.00    87.00
Dian      220101004  2021       76.00    79.00
Eko       220101005  2023       92.00    95.00
Fajar     220101006  2020       88.00    85.00
Gina      220101007  2023       80.00    83.00
Hadi      220101008  2020       82.00    84.00
=====
Nilai UTS Tertinggi: 92.0
Nilai UTS Terendah: 76.0
Nilai rata-rata UAS: 85.38

```

